

Žaidimų rekomendacijų sistema

Kursinio darbo I dalis: projektavimo dokumentas

1. Problema ir idėja

Sistema vienu sakiniu: Žaidimų rekomendacijų sistema, padedanti naudotojui greitai išsirinkti, ką žaisti toliau, pagal jo paties nustatytus filtrus ir svertus.

Problema ir dabartinis procesas: Žmonės renkasi žaidimus naršydami parduotuvių ar žaidimų sąrašų puslapius ir tada renkasi savo nuovoka. Įprastai žaidimų svetainių filtrai taiko tik labai objektyvius kriterijus, neleidžia nurodyti, kas žaidėjui svarbiau, ir nepaaiškina, kodėl vienas žaidimas siūlomas anksčiau už kitą. Be to, naršant svetainėse dažnai siūlomi pasikartojantys ar jau naudotojo anksčiau žaisti žaidimai, o reitingų svetainėse vyrauja specifinių žanrų žaidimų perteklius (pvz.: dažniausiai siūlomi bus singleplayer story-based žaidimai).

Nauda: Naudotojas gauna surikiuotą sąrašą, kuriame prie kiekvieno žaidimo užvedus žymeklį matyti paaiškinimas, kodėl šis žaidimas jam rekomenduojamas. Rezultatą galima keisti keičiant svertus, o sąraše gali būti nerodomi jau žaisti žaidimai ir nėra vieno žanro pertekliaus.

Naudotojai: Registruoti žmonės, kurie mėgsta žaisti vaizdo žaidimus. Jie prisiregistruoja ir prisijungia, peržiūri žaidimų katalogą, pažymi žaidimus savo bibliotekoje (turiu / žaidžiau / nenoriu), nustato filtrus ir svertus ir gauna rikiuotą rekomendacijų sąrašą.

Prielaidos:

1. Kiekvienas žaidimas turi platformą, pagrindinį žanrą, žymes, apytikslę trukmę, žaidėjų režimą ir skaičių, kainą bei kitus kriterijus (nuotaiką / atmosferą, leidimo metus, kūrėją, 2D / 3D).
2. Žaidimų katalogas pradžioje užpildomas vienkartinio importu iš išorinio API ir saugomas duomenų bazėje; trūkstant žaidimai gaunami pagal poreikį. Rekomendacijos skaičiuojamos tik iš duomenų bazėje esančių žaidimų.
3. Kiekvienas naudotojas pats nustato savo svertus.
4. Naudotojas nėra pradedantysis ir jau turi polinkius mėgstamiems žaidimams.
5. Programos algoritmai deterministiškai ir taisyklingai išmatuos naudotojo žaidžiamų žaidimų tendencijas.

2. Apimtis

Funkcija	Ką naudotojas galės atlikti	Pagrindinis modulis ar pagalbinė funkcija
Rekomendacijų generavimas	Nurodyti filtrus bei svertus ir gauti surikiuotą žaidimų sąrašą su paaiškinimais	Pagrindinis modulis
Žaidimų katalogo peržiūra	Naršyti ir ieškoti žaidimų kataloge pagal pavadinimą, žanrą, platformą	Pagalbinė funkcija
Asmeninė biblioteka	Pažymėti žaidimo būseną (turiu / žaidžiau / nenoriu) ir, naudotojo pageidavimu, nebematyti jo kataloge ar rekomendacijose	Pagalbinė funkcija
Paskyros	Prisiregistruoti, prisijungti, išsaugoti savo svertus savo paskyroje ir atsijungti	Pagalbinė funkcija

Į kursinio darbo apimtį neįeina: AI rekomendacijos ar asistentas, socialinės funkcijos (draugai, komentarai, bendri sąrašai), žaidimų pirkimas ir parduotuvių integracijos, mobilioji programa, slaptažodžio atkūrimas ir el. pašto patvirtinimas, nenumatomi API apribojimai (pvz.: tam tikrų platformų kainos nepasiekiamos), periodinė katalogo sinchronizacija su išoriniu API.

3. Pagrindinis modulis

Pavadinimas ir atsakomybė: Rekomendacijų variklis. Iš katalogo parenka žaidimus, atitinkančius naudotojo griežtus filtrus, įvertina juos pagal naudotojo svertus, sudaro surikiuotą sąrašą su rekomendacinių taškų skaičiumi ir paaiškinimu bei tvarko situacijas, kai tinkamų žaidimų nėra.

Logika, kurią reikės projektuoti ir testuoti: Griežtas filtravimas, taškų skaičiavimas pagal keičiamas vertinimo taisykles, deterministinis rikiavimas esant lygiems taškams, žanrų įvairovės ribojimas ir pagalbinių filtrų atlaisvinimas, kai rezultatų nėra.

Įvestis: Pagrindiniai filtrai (griežti): platforma, maksimali trukmė, kainos intervalas, „singleplayer“ ar „multiplayer“ (jei „multiplayer“ – kiek žaidėjų?), žanras, pageidaujami leidimo metai (ar intervalas), ar išskirti žaistus ir turimus žaidimus. Pagalbiniai filtrai (pašalinami, jei kitaip rezultatų nėra): žaidimų įmonė (kūrėjas), 2D / 3D. Svertai: sveikasis skaičius nuo 0 iki 5 (0 – nesvarbu, 5 – labai svarbu), priskiriamas žaidimo žymeii arba nuotakai / atmosferai iš fiksuoto katalogo sąrašo. Taip pat nurodomas norimų pasiūlymų skaičius N ir dar galima nurodyti įvairovės ribą K, kuri rodytų, kiek parodyti vieno žanro žaidimų. Pavyzdys: platforma = PC, trukmė ≤ 12 val., singleplayer; Roguelike = 3, Atmosferiškas = 2, Rekomendacijų sk. N = 3.

Išvestis: Surikiuotas iki N žaidimų sąrašas; kiekvienam nurodomas bendras taškų skaičius ir jo išskaidymas pagal taisykles, taip pat pranešimai apie pašalintus pagalbinius filtrus. Pavyzdys: G1 – 5 tšk. (Roguelike +3, Atmosferiškas +2); G7 – 3 tšk. (Roguelike +3); G3 – 2 tšk. (Atmosferiškas +2).

Veikimo eiga:

1. Gauti užklausą ir ją patikrinti ($N \geq 1$).
2. Iš duomenų bazės paimti kandidatus, atitinkančius griežtus filtrus.
3. Kiekvienam kandidatui apskaičiuoti taškus taikant visas vertinimo taisykles.
4. Surikiuoti pagal taškus ir lygių taškų taisyklę.
5. Pritaikyti įvairovės ribą ir palikti pirmus N.
6. Jei kandidatų nėra, atlaisvinti pagalbinius filtrus ir kartoti nuo 2 žingsnio; jei rezultato vis tiek nėra, grąžinti tuščią sąrašą su priežastimi.

Taisyklės arba sprendimo žingsniai

1. Griežti filtrai. Žaidimas tampa kandidatu tik jei atitinka visus nurodytus pagrindinius filtrus: platformą, trukmė neperžengia nurodytos ribos, kaina ir leidimo metai patenka į nurodytą intervalą, tinka režimas ir žaidėjų skaičius, žanras, ir (jei pasirinkta) nėra pažymėtas naudotojo bibliotekoje kaip žaistas ar turimas.
2. Taškų skaičiavimas. Žaidimo taškai yra sutampančių žymių ar nuotakų / atmosferų svertų suma – kiekviena vertinimo taisyklė realizuoja bendrą sąsają ir grąžina taškus kartu su paaiškinimu.
3. Lygių taškų taisyklė. Esant vienodiems taškams, pirmesnis siūlomas žaidimas su daugiau sutampančių žymių, tada su trumpesne trukme, tada pagal pavadinimą abėcėlės tvarka.
4. Įvairovės riba. Sąrašė negali būti daugiau kaip K žaidimų su tuo pačiu pagrindiniu žanru – perteklinį žaidimą pakeičia kitas pagal taškus.
5. Pagalbinių filtrų šalinimas. Jei kandidatų nėra, pašalinami pagalbiniai filtrai (žaidimų įmonė, 2D / 3D), o pagrindiniai filtrai niekada nekeičiami, o jei rezultatų vis tiek nėra – grąžinamas tuščias sąrašas ir naudotojui pranešama apie nepavykusią paiešką. Jei pagalbiniai filtrai buvo pašalinti rekomendacijos procese, naudotojui irgi apie tai pranešama.

Scenarijai būsimiems testams

Testinis katalogas (visiems scenarijams): G1 – PC, 10 val., Nuotykių, žymės Roguelike + Atmosferiškas; G2 – PC, 15 val., Veiksmo, Roguelike; G3 – PC, 8 val., Nuotykių, Atmosferiškas; G4 – PC, 40 val., RPG, Roguelike; G5 – PS5, 12 val., Veiksmo, Roguelike + Atmosferiškas; G6 – PC, 12 val., Galvosūkių, be svertinių žymių; G7 – PC, 9 val., Veiksmo, Roguelike. Visi žaidimai vienam žaidėjui, naudotojo bibliotekoje žaidimų nėra, svertai visur Roguelike = 3, Atmosferiškas = 2. Kainos, žanro, leidimo metų ir pagalbiniai filtrai scenarijuose nenurodyti.

Scenarijus	Pradinės sąlygos ir konkreti įvestis	Veiksmas	Tikslus laukiamas rezultatas
Įprastas atvejis	Filtrai: PC, ≤ 12 val., Singleplayer; N = 3, K = 2	Paprašyti rekomendacijų	G1 (5 tšk.), G7 (3 tšk.), G3 (2 tšk.) yra siūlomi naudotojui, o G6 (0 tšk.) nepatenka į N; G2 ir G4 atmesti pagal trukmę, G5 – pagal platformą.
Ribinis atvejis arba konfliktas	Filtrai: PC, ≤ 20 val., Singleplayer; N = 2, K = 2	Paprašyti rekomendacijų	G1 (5 tšk.), G7 (3 tšk.) siūlomi naudotojui. G2 taip pat turi 3 tšk., bet ilgesnę trukmę (15 > 9 val.), todėl pagal lygių taškų taisyklę į N nepatenka.
Klaida arba neįmanomas rezultatas	Filtrai: PC, ≤ 6 val., Singleplayer; N = 3, K = 2	Paprašyti rekomendacijų	Tuščias sąrašas ir pranešimas, kad rekomendacijų pagal duotus filtrus nerasta.

Jei modulis naudoja AI: Netaikoma.

4. Kokybės atributas

Pasirinktas atributas: Teisingumas

Kodėl svarbus šiai sistemai: Naudotojas pasitiki rekomendacijomis tik jei jos atitinka jo filtrus ir svertus, o rezultatas yra atkuriamas ir paaiškinamas.

Tikrinimo scenarijus ir sąlygos: Fiksuotas testinis katalogas (3 skyrius) ir užklausų rinkinys su iš anksto apskaičiuotais laukiamais sąrašais ir taškais, įskaitant 3 skyriaus scenarijus.

Sėkmės kriterijus: Ne mažiau kaip 20 scenarijų duoda tiksliai laukiamą žaidimų tvarką ir taškus. Ta pati įvestis 100 kartų iš eilės duoda identišką rezultatą. Joks sąrašo žaidimas neprieštarauja pagrindiniams filtrams ir (jei jie nepašalinti) pagalbiniais.

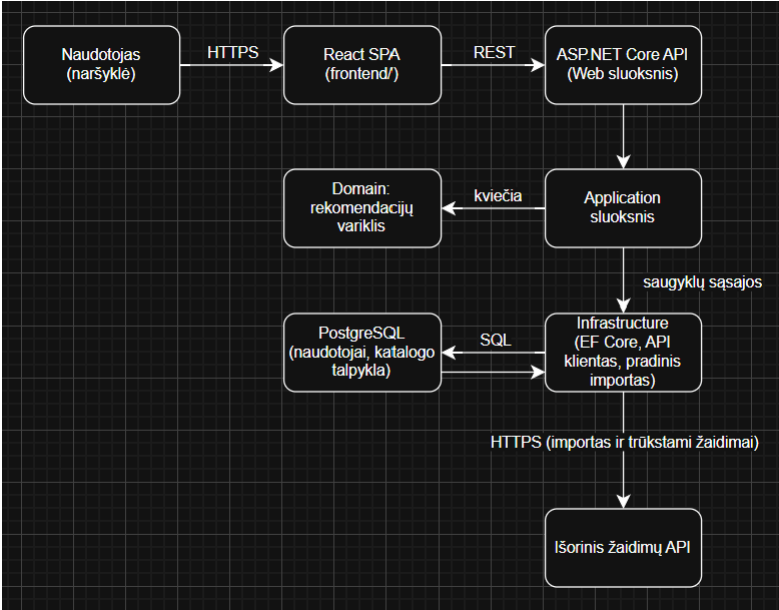
Numatytas projektavimo sprendimas: Deterministinė rikiavimo tvarka su pilna lygių taškų taisykle. Domain logika yra grynai skaičiavimai be išorinės būsenos ir atsitiktinumo. Kartu su rezultatu grąžinamas taškų paaiškinimas.

Kaip patikrinsiu vėlesniame etape: Parametrizuoti unit testai ir integracinis testas su PostgreSQL kontaineriu.

Sprendimo kaina arba ribojimas: Laukiamus rezultatus reikia parengti rankiniu būdu ir sąmoningai keičiant taisykles atnaujinti.

5. Pradinė sistemos struktūra

Paprasta schema



Sistemos dalis	Atsakomybė
React SPA (frontend/)	Naudotojo sąsaja: prisijungimas, katalogas, biblioteka, filtrų ir svertų formos, rekomendacijų sąrašas. Su API bendrauja per REST.
API (Web sluoksnis)	ASP.NET Core Minimal API: užklausų priėmimas, įvesties tikrinimas, autentifikacija, atsakymų formavimas.
Application sluoksnis	Naudojimo scenarijų koordinavimas: per saugyklų sąsajas paima kandidatus ir naudotojo biblioteką, kviečia rekomendacijų variklį.
Domain sluoksnis (rekomendacijų variklis)	Filtrų, vertinimo taisyklių, rikiavimo, įvairovės ir pagalbinių filtrų šalinimo logika. Nepriklauso nuo duomenų bazės, web sluoksnio ar išorinio API.
Infrastructure sluoksnis	EF Core saugyklų realizacijos, migracijos, išorinio API klientas ir vienkartinis pradinis katalogo importas. Trūkstamą žaidimą gauna iš API ir išsaugo duomenų bazėje.
PostgreSQL	Saugo naudotojus, žaidimų katalogą (talpyklą), bibliotekas ir išsaugotus svertus.
Išorinis žaidimų API	Žaidimų duomenų šaltinis pradiniam importui ir trūkstamiems žaidimams. Rekomendacijų variklis jo nekviečia.

Planuojamos technologijos ir pasirinkimo priežastys: C# ir .NET 10 (LTS) su ASP.NET Core: stipri tipizacija palengvina taisyklių modelį ir testavimą, be to, motyvuoja noras praktikuotis rečiau naudojamas technologijas. React + TypeScript (Vite) atskirame kataloge frontend/ toje pačioje repozitorijoje: praktiškas pasirinkimas šiuolaikinėje IT industrijoje, plačiai naudojamas, todėl jo naudojimas stiprina mano kaip IT studento praktines kompetencijas. PostgreSQL + EF Core: ryšiai tarp žaidimų, žanrų ir žymių bei indeksai filtravimui. Autentifikacija: ASP.NET Core Identity. Išorinio API klientas: HttpClient. Testai: xUnit ir Testcontainers (backend), Vitest (frontend). Docker Compose (API, frontend, PostgreSQL) ir GitHub Actions testų paleidimui.

6. AI panaudojimas

AI rengiant šį dokumentą

Taip, naudoju AI (Claude, Anthropic).

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
Claude: projekto idėjos paieška ir atranka	Pasiūlyti projektų idėjų sąrašai; žaidimų rekomendacijų sistemos taisyklių struktūra (filtrai, taškai, įvairovė, tuščio rezultato tvarkymas).	Atmečiau idėjas su AI modulių ir idėjų grupes, kurios nesudomino. Sugalvojau žaidimų rekomendacijų sistemą ir pritaikiau ją savo reikalavimams (griežti filtrai, konkretūs svertai pagal nuotaiką / atmosferą, pagalbiniai filtrai), nes noriu prisiimti atsakomybę už savo projektą.	Kiekvieną pasiūlymą peržiūrėjau ir taisiau savo pastabomis, o mano idėjai prieštaraujančias taisykles (pvz., filtrų atlaisvinimą) perrašiau.
Claude: technologijų parinkimas	Pasiūlytas C# / .NET, PostgreSQL, testavimo priemonių rinkinys ir kokybės atributai.	Blazor pakeičiau į React, nes jis praktiškesnis dabartinėje IT industrijoje. Pasiūlytą palaikomumą kaip pagrindinį atributą pakeičiau teisingumu, nes nauji kriterijai keičia rezultatus ir palaikomumo kriterijus tapo per sudėtingas. Katalogo pildymą pakeičiau į pradinį importą su papildymu pagal poreikį, nes taip atrodė efektyviausia.	Pasirinkimus tikslinau pagal savo reikalavimus ir sistemos struktūrą (diagrama pačio braižyta).
Claude: šio dokumento dalinis pildymas	Pirminis užpildyto dokumento juodraštis, testinis katalogas, bei testavimo scenarijai.	Perrašiau ir pataisiau visą dokumentą savo formuluotėmis ir idėjomis, keičiau programos veikimo taisykles ir griežtai analizavau sugeneruotą tekstą kad galų gale dokumentas būtų pilnai perdarytas mano rankomis.	Perskaičiau visą dokumentą ir taisiau pagal savo supratimą apie sistemą.

Planuojamas AI naudojimas kuriant sistemą

Kur ir kam naudosiu AI: Kodo ir testų pradinėms stadijoms (pvz., taisyklių klasėms, xUnit testams, API endpoint'ų karkasui), dokumentacijos formuluotėms ir klaidų paieškai.

Kaip tikrinsiu pasiūlymus ir sugeneruotą kodą: Kiekvieną sugeneruotą kodo dalį peržiūrėsiu rankiniu būdu, kad galėčiau ją paaiškinti, ir prieš priimdamas paleisiu testus; neatitinkančius taisyklių ar testų rezultatų nepriimsiu arba perrašysiu.

Ar AI bus sistemos funkcionalumo dalis: Ne. Rekomendacijos grindžiamos tik deterministinėmis taisyklėmis.

7. Tolesnių darbų planas

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
Domain sluoksnis ir taisyklės	Filtravimas, taškų skaičiavimas, rikiavimas, įvairovės riba ir pagalbinių filtrų šalinimas su unit testais ant 7 žaidimų testinio katalogo	1
Duomenų sluoksnis ir katalogo importas	EF Core modelis, migracijos, PostgreSQL konteineryje, išorinio API klientas ir pradinis katalogo importas	2

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
API ir paskyros	Registracija, prisijungimas, katalogo, bibliotekos ir rekomendacijų endpoint'ai	3
React sąsaja	Prisijungimo, katalogo, bibliotekos, filtrų ir svertų formų bei rekomendacijų sąrašo puslapiai	4
Testai	Teisingumo rinkinys (≥ 20 scenarijų) ir integracinis testas su PostgreSQL	5

Būsimo prototipo veikimo scenarijus: Prisijungęs naudotojas pasirenka PC, trukmę ≤ 12 val., singleplayer, svertus Roguelike = 3 ir Atmosferiškas = 2, N = 3 ir paprašo rekomendacijų. Duomenys: 3 skyriuje aprašytas testinis katalogas. Laukiamas rezultatas: G1 (5 tšk.), G7 (3 tšk.), G3 (2 tšk.) su taškų paaiškinimu; pakeitus trukmę į ≤ 6 val., rodomas pranešimas, kad rekomendacijų pagal duotus filtrus nerasta.

Rizika arba neaiškumas	Kaip patikrinsiu arba sumažinsiu
Išorinio API limitai, nepasiekiamumas ar trūkstami laukai (trukmė, žymės) gali sutrukdyti katalogo importui	Importuosiu dalį katalogo; rekomendacijos veiks tik iš duomenų bazės; testuose naudosis fiksuotą testinį katalogą be tikrų API užklausų.
Išorinis API gali neturėti nuotaikos / atmosferos duomenų	Prieš prototipą patikrinsiu, kokias žymes pateikia API, ir sudarysiu fiksuotą nuotaikų sąrašą su žymių atitikimu; jei duomenų per mažai, naudosis tik žymes.
Svertai gali duoti nepatenkinamus rezultatus	Svertai konfigūruojami, taškų išskaidymas rodomas rezultate; prototipe palyginsiu rezultatus su nedideliu realių žaidimų pavyzdžiu ir koreguosiu.